Point elevations from survey differences

Gabriel Arellano

2026-11-12

Contents

1 Purpose 1

2 Read the survey 2

3 Estimate elevations 5

4 Comparison with Condit's solver 5

5 Small checks for the old problems 8

1 Purpose

This solver estimates one elevation per point from measured height differences between pairs of points. It follows Richard Condit's equal-weight least-squares calculation developed for BCI plot and other ForestGEO sites.

Height differences can come from a regular grid of stakes after the establishment of a plot. A survey team measures the vertical difference between pairs of stakes, with different instruments and practical procedures. The coordinates can also describe other surveyed locations, not necessarily stakes in a regular grid. The solver works on irregularly spaced points and does not require a complete regular grid. See Condit's survey example for an overview of the process, using the old tools and conventions.

This is a limited tool for pairwise survey records, with little consideration for error propagation and so forth. For a new survey, professional teams or drone mapping could provide a more detailed elevation model.

2 Read the survey

This section reads a simulated survey as an example. It follows mostly a regular grid, with some local complications and unusable points that were by-passed by taking measurements around.

The default paths below work when rendering this tutorial or running from the repository root. Set a full path to fgeo2 when using the code elsewhere.

```
# install.packages(c("Matrix", "igraph", "jsonlite"))
path.to.fgeo2 = if(file.exists("../../fgeo2.Rproj")) "../.." else "."
#path.to.fgeo2 = "/Users/salva/Dropbox/PERSONAL/LLC/Oikobit LLC/projects/code-and-statisti

# Coordinates and height differences are in metres.
records <- read.csv(file.path(path.to.fgeo2, "data_examples/topography-survey.csv"))

# One row per measured difference. Positive dz means the destination is higher.
# Of course, you would need to adapt this code to the shape of your stored data.
survey <- data.frame(
    x_i = records$from_x_m,
    y_i = records$from_y_m,
    x_j = records$to_x_m,
    y_j = records$to_y_m,
    dz = records$height_difference_m
)
head(survey)
```

```
  x_i y_i x_j y_j     dz
1   0   0   0  20 0.2971
2   0   0  20   0 1.5565
3   0  20   0  40 0.2978
4   0  20  20  20 1.5266
5   0  40   0  60 0.2974
6   0  40  20  40 1.3458
```

```
# Horizontal distances between the two surveyed locations.
survey.distance <- sqrt((survey$x_j - survey$x_i)^2 + (survey$y_j - survey$y_i)^2)
table(survey.distance)
```

```
survey.distance
               5               10 11.1803398874989  14.142135623731
```

```
               3                14                 2                10
              15 15.8113883008419                20 20.6155281280883
               5                 5               284                 6
22.3606797749979 25.4950975679639                30 31.6227766016838
               1                 2                 2                 2
```

```r
# Fraction removed from each arrow: 0 = full length, 0.3 = shorten by 30%.
arrow.reduction = 0.2

# Planned grid and measured locations.
grid <- expand.grid(x = seq(0, 240, 20), y = seq(0, 240, 20))
measured <- unique(rbind(
    data.frame(x = survey$x_i, y = survey$y_i),
    data.frame(x = survey$x_j, y = survey$y_j)
))

# Unusable grid points are absent from both sets of survey endpoints.
unusable <- !paste(grid$x, grid$y) %in% paste(measured$x, measured$y)

# Shorten each arrow while keeping its origin and direction.
arrow.x <- survey$x_i + (1 - arrow.reduction) * (survey$x_j - survey$x_i)
arrow.y <- survey$y_i + (1 - arrow.reduction) * (survey$y_j - survey$y_i)

# One pane, with equal horizontal and vertical scales.
settings <- par(mfrow = c(1, 1), mar = c(4, 4, 1, 1))
plot(grid$x, grid$y, type = "n", asp = 1,
     xlim = c(0, 240), ylim = c(0, 240),
     xlab = "x (m)", ylab = "y (m)")

# Full grid
abline(v = seq(0, 240, 20), h = seq(0, 240, 20), col = "grey85", lwd = 0.5)

# Survey arrows
arrows(survey$x_i, survey$y_i, arrow.x, arrow.y,
       length = 0.03, angle = 25, code = 2, col = "black", lwd = 0.7)

# Surveyed locations, including those on the finer 5 m grid.
points(measured$x, measured$y, cex = 0.3, col = "black", pch = 16)

# Grid points
points(grid$x, grid$y, cex = 0.4, col = "black", pch = 16)
points(grid$x[unusable], grid$y[unusable], cex = 0.9, col = "firebrick", pch = 4, lwd = 1.
```

```
par(settings)
```

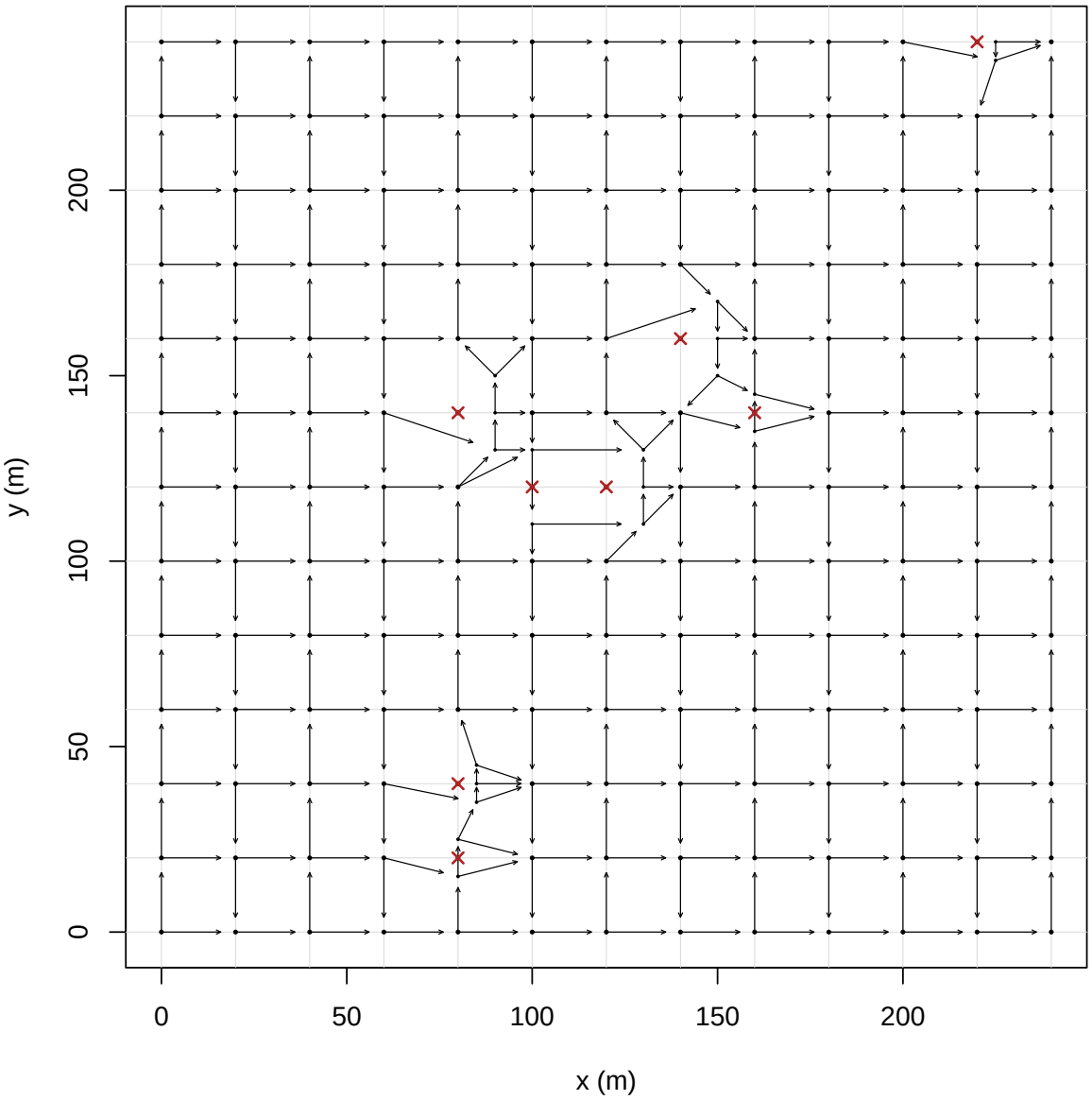


Figure 1: Simulated survey directions on the planned 20 m grid. Red crosses mark unusable grid points.

3 Estimate elevations

This sections shows how the new topography solver works. It is just one function and its use is trivial, once the "survey" input has the needed five columns with the right column names. Set the coordinates and elevation of the reference point, then fit all measured differences together.

```
# Assign this elevation to the point at these coordinates.
reference.xy = c(0, 0)
reference.z = 371  # m.a.s.l. Use 0 for relative elevations if absolute elevation is unkno

source(file.path(path.to.fgeo2, "06_topography_habitat/R/new/topography-solver.R"))
elevations <- estimate_point_elevations(survey, reference.xy, reference.z)
head(elevations)
```

```
  x   y        z
1 0   0 371.0000
2 0  20 371.2909
3 0  40 371.5931
4 0  60 371.8797
5 0  80 372.1822
6 0 100 372.4800
```

Other file layouts must be processed into the same five-column `survey` table. The solver identifies points by exact coordinate pairs, so use consistent coordinates for repeated points. The point IDs in this example file are only used by the old solver below.

Rows and columns can be in any order. Repeated and conflicting measurements receive equal weight. Incomplete rows are omitted with a warning. Missing measurements or missing grid points are acceptable while the remaining observations connect every retained point. Points represented only by omitted rows disappear from the output.

The result contains `x`, `y` and `z`, as the most general output. Raster construction and interpolation, if needed, would be separate steps.

4 Comparison with Condit's solver

Condit solver requires three functions and has known issues (see evaluation file and checks below). But it contains the core calculations and intention: the new solver must match the results from the old one. In the comparison below, the old code is read from fgeo2 as it stood

through 2026-11-09, before this replacement. This avoids depending on copies in the current `R/old` or `evaluation/Condit` folders. The date lookup needs internet access and `jsonlite`.

```
source(file.path(path.to.fgeo2, "evaluation/0-code-to-source.r"))
old <- new.env()
load_github_snapshot(
    commit.date = "2026-11-09",
    scripts = "06_topography_habitat/R/new/topography-solvetopo.r",
    envir = old
)
```

```
gabrielareto/fgeo2 commit: 0a2c01dd560daacba0fdc2eb8c9280e2b9d2ea9d
```

```
# If needed, replace the call above with one alternative below.
# source(file.path(path.to.fgeo2, "06_topography_habitat/R/old/topography-solvetopo.r"), l
# source(file.path(path.to.fgeo2, "evaluation/Condit/topography/solvetopo.r"), local = old
# old$solve.topo <- ctfs::solve_topo  # Requires the ctfs package.
```

First compare relative elevations at datum zero. Both solvers should reconcile the measurements in the same way.

```
# Condit's solver needs point IDs and a coordinate lookup.
points <- unique(rbind(
    data.frame(id = records$from_point, x = records$from_x_m, y = records$from_y_m),
    data.frame(id = records$to_point, x = records$to_x_m, y = records$to_y_m)
))
points <- points[order(points$x, points$y), ]
base.id <- points$id[points$x == reference.xy[1] & points$y == reference.xy[2]]

old.survey <- data.frame(
    pt1 = records$from_point,
    pt2 = records$to_point,
    htdiff = records$height_difference_m
)
old.fit <- old$solve.topo(old.survey, basept = base.id, baseelev = 0)
```

```
at point  1
at point  101
```

```
old.z <- old.fit$elev[match(points$id, old.fit$label)]
new.fit <- estimate_point_elevations(survey, reference.xy, reference.z = 0)

# Align by coordinates, then check agreement.
stopifnot(all(points$x == new.fit$x), all(points$y == new.fit$y))
max(abs(old.z - new.fit$z)) # minor floating point disagreements
```

```
[1] 1.012523e-13
```

```
stopifnot(isTRUE(all.equal(old.z, new.fit$z, tolerance = 1e-8)))
```

The two panels show the topography estimated from both solvers. For visualization purposes, the {x, y, z} output of the solvers has been interpolated into a complete 5x5m grid. The z dimension has been exaggerated for better visualization by the same factor. Both panels use the same scales and viewing angles, using R perspective plots.

```
# install.packages("interp")

x <- seq(0, 240, 5)
y <- seq(0, 240, 5)

# Interpolate all points, including those outside the main 20 m grid.
old.surface <- interp::interp(points$x, points$y, old.z,
                              xo = x, yo = y, method = "linear")$z
new.surface <- interp::interp(new.fit$x, new.fit$y, new.fit$z,
                              xo = x, yo = y, method = "linear")$z

# Vertical exaggeration for display.
zoomz = 3
z.limits <- range(old.surface, new.surface, na.rm = TRUE) * zoomz

settings <- par(mfrow = c(1, 2), mar = c(1, 1, 3, 1))

persp(x, y, old.surface * zoomz, zlim = z.limits, theta = 35, phi = 25,
      scale = FALSE, col = "lightblue", shade = 0.4,
      xlab = "x (m)", ylab = "y (m)", zlab = "z (m)", main = "Condit")
persp(x, y, new.surface * zoomz, zlim = z.limits, theta = 35, phi = 25,
      scale = FALSE, col = "lightblue", shade = 0.4,
      xlab = "x (m)", ylab = "y (m)", zlab = "z (m)", main = "New solver")

par(settings)
```

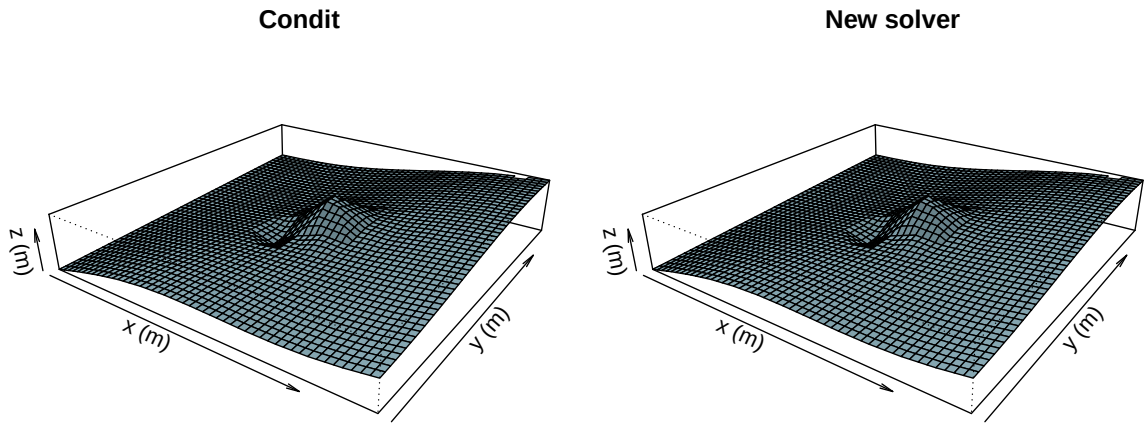


Figure 2: Elevations interpolated onto a 5 m grid. Both solvers agree.

5 Small checks for the old problems

A chain with differences of 2 and 3 m should give elevations 100, 102 and 105 m when its first point is at 100 m. The old code applies the datum only to the first point. It also mishandles reordered columns and propagates a missing difference. These small examples show the fixes.

```
chain <- data.frame(x_i = c(0, 1), y_i = 0, x_j = c(1, 2), y_j = 0, dz = c(2, 3))
old.chain <- data.frame(pt1 = c("1", "2"), pt2 = c("2", "3"), htdiff = c(2, 3))

# A nonzero datum must shift every point.
rbind(
    Condit = old$solve.topo(old.chain, baseelev = 100)$elev,
    New = estimate_point_elevations(chain, reference.z = 100)$z
)
```

```
at point  1

       [,1] [,2] [,3]
Condit  100    2    5
New     100  102  105
```

```r
  # Reordering columns must preserve the direction of the measured differences.
  rbind(
      Condit = old$solve.topo(old.chain[c("pt2", "pt1", "htdiff")])$elev,
      New = estimate_point_elevations(chain[c("dz", "x_j", "y_j", "x_i", "y_i")])$z
  )
```

```
at point  1
```

```
       [,1] [,2] [,3]
Condit    0   -2   -5
New       0    2    5
```

```r
  # An extra row with a missing difference must not contaminate usable measurements.
  rbind(
      Condit = old$solve.topo(rbind(old.chain, transform(old.chain[1, ], htdiff = NA)))$elev
      New = estimate_point_elevations(rbind(chain, transform(chain[1, ], dz = NA)))$z
  )
```

```
at point  1

Warning: 1 incomplete or same-point observation(s) omitted.
```

```
       [,1] [,2] [,3]
Condit    0   NA   NA
New       0    2    5
```